

Q1.

- In standard SQL, NULL represents an "unknown" or missing value. Any direct comparison with NULL using equality (e.g., `phone_number = NULL`) evaluates to UNKNOWN rather than TRUE, causing the WHERE clause to filter out all rows.
- **Corrected Query:** `SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;`
- NULL is excluded from most aggregate computations (like AVG or SUM) and comparison operations because propagating an unknown value mathematically yields an unknown result, which would render aggregated totals meaningless

Q2.

- Yes, the DISTINCT keyword is redundant here.
- **Justification:** The `GROUP BY department` clause already ensures that each department appears only once in the final result set. DISTINCT executes after the grouping, attempting to remove duplicates from an output that is already guaranteed to be unique per row.

Q3.

- The difference in row count indicates that there are records in the `Customer` table that do not have any corresponding records in the `Order` table.
- If a customer has registered an account but has never placed an order, an `INNER JOIN` will drop them from the results, whereas a `LEFT JOIN` will retain them (filling the order columns with NULLs), yielding a higher overall row count.

Q4.

- The query is invalid because `employee_name` is selected but is neither wrapped in an aggregate function nor included in the `GROUP BY` clause. Conceptually, if SQL allowed this, it would not know *which* employee's name to display alongside the department's single aggregated average salary.
- **Corrected Query:** `SELECT department, AVG(salary) FROM Employee GROUP BY department;`

Q5.

- **Error:** The `WHERE` clause filters rows *before* grouping occurs, so it cannot be used to evaluate aggregate functions like `AVG(salary)`.
- **Corrected Query:** `SELECT department, AVG(salary) FROM Employee GROUP BY department HAVING AVG(salary) > 80000;`

Q6.

- **(a) Non-correlated:** The overall average product price is a single, static value that is independent of individual product rows, so the subquery only needs to execute once.
- **(b) Correlated:** The inner query must reference the department of the *current* row being evaluated by the outer query to calculate that specific department's average.
- **Performance Implication:** Correlated subqueries are generally much slower on large tables because they must be executed repeatedly (once for every single row returned by the outer query), whereas non-correlated subqueries execute only a single time.

Part II: Long Answer Question

(a) Customers with no orders

- **Query:** `SELECT name FROM Customer WHERE customer_id NOT IN (SELECT customer_id FROM "Order");`
- **Justification:** I utilised a `NOT IN` subquery because it cleanly and explicitly expresses the set-subtraction logic required to capture "never". It directly isolates customers entirely

absent from the `Order` table without relying on the indirect `NULL` checks required by an outer join.

(b) Average order value per restaurant:

```
WITH OrderTotals AS (  
    SELECT o.restaurant_id, o.order_id, SUM(m.price * oi.quantity)  
as order_val  
    FROM "Order" o  
    JOIN OrderItem oi ON o.order_id = oi.order_id  
    JOIN MenuItem m ON oi.item_id = m.item_id  
    GROUP BY o.restaurant_id, o.order_id  
)  
SELECT restaurant_id, AVG(order_val) as avg_val  
FROM OrderTotals  
GROUP BY restaurant_id  
HAVING COUNT(order_id) > 5;
```

Justification: I used a Common Table Expression (CTE) to first compute the total value of each individual order, and then aggregated those values per restaurant. `HAVING` was used to enforce the "> 5 orders" condition because filtering by an aggregate count must happen after the grouping phase

(c) Restaurants exceeding platform-wide average:

```
WITH OrderTotals AS (  
    SELECT o.restaurant_id, o.order_id, SUM(m.price * oi.quantity)  
as order_val  
    FROM "Order" o  
    JOIN OrderItem oi ON o.order_id = oi.order_id  
    JOIN MenuItem m ON oi.item_id = m.item_id  
    GROUP BY o.restaurant_id, o.order_id  
) ,  
RestaurantAverages AS (  
    SELECT restaurant_id, AVG(order_val) as avg_val  
    FROM OrderTotals  
    GROUP BY restaurant_id  
    HAVING COUNT(order_id) > 5  
)  
SELECT restaurant_id  
FROM RestaurantAverages  
WHERE avg_val > (SELECT AVG(order_val) FROM OrderTotals);
```

Justification: This requires a non-correlated subquery to determine the platform-wide average because that average is a static, global metric. A correlated subquery is unnecessary and would harm performance, as the platform average does not change based on which restaurant is being evaluated.

(d) Most ordered menu item per restaurant

```

WITH ItemTotals AS (
    SELECT m.restaurant_id, oi.item_id, SUM(oi.quantity) as
total_qty
    FROM OrderItem oi
    JOIN MenuItem m ON oi.item_id = m.item_id
    GROUP BY m.restaurant_id, oi.item_id
)
SELECT restaurant_id, item_id
FROM ItemTotals t1
WHERE total_qty = (
    SELECT MAX(total_qty)
    FROM ItemTotals t2
    WHERE t1.restaurant_id = t2.restaurant_id
);

```

Justification: I used a CTE to pre-aggregate quantities and a correlated subquery to isolate the local maximums. A primary limitation of a basic `GROUP BY + MAX` approach is that while it retrieves the highest numerical value, it cannot directly output the unaggregated `item_id` associated with that max value without this type of self-referencing subquery or window function.

Part III: Normalisation

Q1. Functional Dependencies Based on the scenario rules:

- `student_id -> student_name`
- `course_id -> course_name, instructor_id`
- `instructor_id -> instructor_name, instructor_office`
- `(student_id, course_id) -> grade, enrollment_date`

Q2. 1NF Check Yes, the table is in 1NF. We assume all attributes in this relational design contain single, atomic values and there are no repeating groups nested within rows.

Q3. Partial Dependency (Violating 2NF)

- **Dependency:** `student_id -> student_name` and `course_id -> course_name, instructor_id` are partial dependencies because they rely on only a fraction of the composite primary key (`student_id, course_id`).
- **Anomaly Example:** This causes an insertion anomaly. We cannot add a newly created course (and its instructor details) into the database until at least one student enrolls in it, because `student_id` is a mandatory part of the primary key.

Q4. Transitive Dependency (Violating 3NF)

- **Dependency:** `course_id` $\rightarrow$ `instructor_id` $\rightarrow$ `instructor_office` is a transitive dependency (an attribute depends on another non-key attribute).
- **Anomaly Example:** This causes an update anomaly. If an instructor moves to a new office, the system must update the `instructor_office` field across every single enrollment record for every course they teach; missing even one creates contradictory data.

Q5. 3NF Decomposition Decomposed into 3NF:

- **Student** (`student_id`, `student_name`)
 - *Justification:* Separates the partial dependency on `student_id`.
- **Instructor** (`instructor_id`, `instructor_name`, `instructor_office`)
 - *Justification:* Separates the transitive dependency cascading from `instructor_id`.
- **Course** (`course_id`, `course_name`, `instructor_id`)
 - *Justification:* Separates the partial dependency on `course_id` while maintaining the link to the instructor.
- **Enrollment** (`student_id`, `course_id`, `grade`, `enrollment_date`)
 - *Justification:* Retains the attributes fully functionally dependent on the entire composite primary key.

Part IV: Design Justification

In Part II(a), I chose to use a `NOT IN` subquery to locate customers with zero orders. While a `LEFT JOIN` paired with a `WHERE order_id IS NULL` check achieves the exact same output, I opted for the subquery purely for readability and intent. The subquery cleanly translates the English requirement—"never placed an order"—into straightforward set-subtraction logic, making the code's objective immediately obvious.

Regarding Part III, normalising the `Enrollment` table into 3NF is critical to eliminate severe update and insertion anomalies, but it heavily trades off query convenience. Once normalised, pulling a full student transcript that includes course titles and instructor offices requires an expensive four-way join across `Student`, `Enrollment`, `Course`, and `Instructor`. This elevates database computational load. Real-world systems sometimes intentionally preserve denormalisation—such as storing `course_name` directly in `Enrollment`—to optimise read performance for heavy reporting tools, strategically accepting minor redundancy risks to avoid continuous, costly joins.